03-1-build-release-pipeline

accounts-api security assessment · generated from the repository, not hand-edited

```
# 03-1-build-release-pipeline.yaml
#
# Section 3 deliverable: the pipeline implementing SBOM attestation, image
# signing, OIDC auth and immutable tags.
#
# Assembled by scripts/build-submission.py from the repository.

# ============================================================================
# SOURCE: .github/workflows/build-release.yml
# ============================================================================
# Section 2 — build and release integrity.  Threats: T1 (CI/CD identity abuse), T4
(unsigned/mutable artefact).
#
# THIS WORKFLOW ACTUALLY RUNS. Push this repo to GitHub and the Actions tab shows
# a green run that really builds the image, really generates an SBOM, really
# keyless-signs and attests it against Sigstore, and then really verifies its own
# output before anything is called releasable. No AWS account and no secrets are
# required for that path — it targets GHCR, which is free.
#
# The AWS half (ECR push, EKS rollout) is written exactly as it would ship and is
# gated behind the repository variable AWS_ENABLED. Set it to "true" in a repo
# that has the roles from iam/ and the same jobs run against ECR and EKS. Left
# unset — the default — those steps are skipped and say so in the job summary,
# rather than failing and leaving a red pipeline nobody trusts.
#
# Design rules enforced here:
#   1. No static registry or cloud credentials. OIDC / GITHUB_TOKEN only.  (T1)
#   2. Every third-party action pinned to a full commit SHA. A tag is mutable; a
#      compromised maintainer re-points v4 and owns our deploy role.  (T1)
#   3. Least-privilege `permissions` at workflow level, widened per job.  (T1)
#   4. The image is pushed and then referenced ONLY by digest.  (T4)
#   5. The pipeline verifies its own signature and attestation before deploy, and
#      that verification is hardened against the Sigstore availability stall that
#      makes naive `cosign verify` jobs hang (see docs/02-build-integrity.md).
name: build-release

on:
  push:
    branches: [main]
    paths-ignore: ["**/*.md", "docs/**"]
  workflow_dispatch:

# NOTE: deliberately NOT `pull_request_target`. That trigger runs with the base
# repo's secrets and OIDC identity against fork-controlled code — it is the single
# most common route to T1 and there is no safe way to use it in this repo.

permissions: {}          # default deny; each job opts in to exactly what it needs

concurrency:
  group: build-release-${{ github.ref }}
  cancel-in-progress: false

env:
  AWS_REGION: ap-south-1
  ECR_ACCOUNT: "210987654321"                       # build account
  IMAGE_NAME: accounts-api
```

```
jobs:
  # ---------------------------------------------------------------------------
  build:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      id-token: write            # OIDC: Fulcio, and AWS when AWS_ENABLED
      packages: write            # GHCR
    outputs:
      digest: ${{ steps.push.outputs.digest }}
      image: ${{ steps.meta.outputs.image }}
      version: ${{ steps.meta.outputs.version }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with:
          persist-credentials: false     # don't leave a push-capable token on disk

      # Immutable, versioned tag: <semver>-<short-sha>. Recomputed from git, never
      # supplied by a PR author. `latest` is never produced by this pipeline.
      - id: meta
        run: |
          set -euo pipefail
          VERSION="$(cat VERSION)-${GITHUB_SHA::12}"
          if [ "${{ vars.AWS_ENABLED }}" = "true" ]; then
            IMAGE="${ECR_ACCOUNT}.dkr.ecr.${AWS_REGION}.amazonaws.com/${IMAGE_NAME}"
          else
            IMAGE="ghcr.io/${GITHUB_REPOSITORY_OWNER,,}/${IMAGE_NAME}"
          fi
          echo "version=$VERSION" >> "$GITHUB_OUTPUT"
          echo "image=$IMAGE"     >> "$GITHUB_OUTPUT"
          {
            echo "### Build target"
            echo ""
            echo "| | |"
            echo "|---|---|"
            echo "| image | \`$IMAGE\` |"
            echo "| version tag | \`$VERSION\` |"
            echo "| AWS path | ${{ vars.AWS_ENABLED == 'true' && 'ENABLED (ECR + EKS)' || 'skipped
— set repo variable AWS_ENABLED=true to exercise it' }} |"
          } >> "$GITHUB_STEP_SUMMARY"

      # ---- AWS path (production) ---------------------------------------------
      - if: vars.AWS_ENABLED == 'true'
        uses: aws-actions/configure-aws-credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502 #
v4.0.2
        with:
          role-to-assume: arn:aws:iam::210987654321:role/gha-accounts-api-ecr-push
          aws-region: ${{ env.AWS_REGION }}
          role-session-name: gha-${{ github.run_id }}-${{ github.run_attempt }}
          role-duration-seconds: 900          # 15 min; the job takes ~6

      - if: vars.AWS_ENABLED == 'true'
        uses: aws-actions/amazon-ecr-login@062b18b96a7aff071d4dc91bc00c4c1a7945b076 # v2.0.1

      # Belt and braces on top of ECR's IMMUTABLE setting (terraform/ecr.tf): fail
      # loudly rather than silently no-op if this tag already exists.  (T4)
      - name: Refuse to reuse an existing tag
        if: vars.AWS_ENABLED == 'true'
        run: |
          set -euo pipefail
          if aws ecr describe-images --repository-name "$IMAGE_NAME" \
               --image-ids imageTag="${{ steps.meta.outputs.version }}" >/dev/null 2>&1; then
            echo "::error::tag ${{ steps.meta.outputs.version }} already exists — refusing to
mutate"
```

```
            exit 1
          fi

      # ---- GHCR path (default; free, and what makes this workflow runnable) --
      - if: vars.AWS_ENABLED != 'true'
        uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567 # v3.3.0
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      # GHCR has no registry-side tag-immutability setting, which is a genuine
      # downgrade from ECR's IMMUTABLE and is recorded as such in DECISIONS.md.
      # This check is the compensating control on the free path.
      - name: Refuse to reuse an existing tag (GHCR)
        if: vars.AWS_ENABLED != 'true'
        continue-on-error: true      # first-ever build has no package to query
        run: |
          set -euo pipefail
          REF="${{ steps.meta.outputs.image }}:${{ steps.meta.outputs.version }}"
          if docker manifest inspect "$REF" >/dev/null 2>&1; then
            echo "::error::tag $REF already exists — refusing to mutate"
            exit 1
          fi
          echo "tag is free"

      - uses: docker/setup-buildx-action@c47758b77c9736f4b2ef4073d4d51994fabfe349 # v3.7.1

      - id: push
        uses: docker/build-push-action@48aba3b46d1b1fec4febb7c5d0c644b249a11355 # v6.10.0
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.image }}:${{ steps.meta.outputs.version }}
          provenance: mode=max        # SLSA v1 build provenance, attached to the index
          sbom: false                 # we produce our own with syft, below
          cache-from: type=gha
          cache-to: type=gha,mode=min # `min` — build caches are a cross-PR data path

  # ------------------------------------------------------------------------
  # SBOM + signature. Separate job so the signing identity in the Fulcio
  # certificate is this job, and so a reviewer can read the trust chain in one place.
  attest:
    needs: build
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      id-token: write             # Fulcio: exchanges the GH OIDC token for a short-lived cert
      packages: write
    env:
      REF: ${{ needs.build.outputs.image }}@${{ needs.build.outputs.digest }}
    steps:
      - uses: sigstore/cosign-installer@dc72c7d5c4d10cd6bcb8cf6e3fd625a9e5e537da # v3.7.0

      - if: vars.AWS_ENABLED == 'true'
        uses: aws-actions/configure-aws-credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502 #
v4.0.2
        with:
          role-to-assume: arn:aws:iam::210987654321:role/gha-accounts-api-ecr-push
          aws-region: ${{ env.AWS_REGION }}
          role-session-name: attest-${{ github.run_id }}
      - if: vars.AWS_ENABLED == 'true'
        uses: aws-actions/amazon-ecr-login@062b18b96a7aff071d4dc91bc00c4c1a7945b076 # v2.0.1
      - if: vars.AWS_ENABLED != 'true'
```

```
        uses: docker/login-action@9780b0c442fbb1117ed29e0efdff1e18412f7567 # v3.3.0
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      # SBOM is generated from the *pushed image by digest*, not from the workspace,
      # so it describes the artefact that will actually run.
      - uses: anchore/sbom-action@df80a981bc6edbc4e220a492d3cbe9f5547a6e75 # v0.17.9
        with:
          image: ${{ env.REF }}
          format: spdx-json
          output-file: sbom.spdx.json
          upload-artifact: false      # deliberate: see below

      # Pre-warm the Sigstore TUF trust root ONCE, with a hard OS-level timeout.
      # Rationale: cosign's own --timeout does not bound the TUF/network setup
      # phase, so a slow public-good endpoint hangs the job rather than failing it.
      # Every cosign call below is likewise wrapped in `timeout` with retries, so
      # this job either completes, rides out a transient stall, or fails fast and
      # blocks the release. It never hangs.
      - name: Pin and pre-warm the Sigstore trust root
        run: timeout 120 cosign initialize --mirror https://tuf-repo-cdn.sigstore.dev --root
https://tuf-repo-cdn.sigstore.dev/root.json

      # The SBOM is published as an in-toto attestation attached to the image digest
      # in the registry — NOT as a build-log artefact.  (T4)
      # Rationale: Actions artefacts expire (90 days default), are not addressable by
      # digest, and are not retrievable by an incident responder holding only a
      # running container's image reference. `cosign download attestation` is.
      - name: Sign image and attach SBOM attestation (keyless)
        run: |
          set -euo pipefail
          retry() { for i in 1 2 3; do timeout 180 "$@" && return 0; echo "::warning::attempt $i
failed: $*"; sleep $((i*10)); done; return 1; }
          retry cosign sign --yes "$REF"
          retry cosign attest --yes --type spdxjson --predicate sbom.spdx.json "$REF"

      # Verification is part of the pipeline, not a claim in a README.  (T4)
      # The identity is computed rather than hard-coded so this works in a fork
      # without editing the workflow — the value it checks is still the exact
      # workflow ref, which is the property that matters.
      - name: Verify signature and attestation
        run: |
          set -euo pipefail
          IDENTITY="https://github.com/${GITHUB_REPOSITORY}/.github/workflows/build-
release.yml@${GITHUB_REF}"
          ISSUER="https://token.actions.githubusercontent.com"
          retry() { for i in 1 2 3; do timeout 180 "$@" && return 0; echo "::warning::attempt $i
failed"; sleep $((i*10)); done; return 1; }

          retry cosign verify \
            --certificate-identity "$IDENTITY" \
            --certificate-oidc-issuer "$ISSUER" \
            "$REF" > verify-signature.json

          retry cosign verify-attestation --type spdxjson \
            --certificate-identity "$IDENTITY" \
            --certificate-oidc-issuer "$ISSUER" \
            "$REF" > /dev/null

          {
            echo "### Supply chain"
            echo ""
```

```
            echo "- artefact: \`$REF\`"
            echo "- signed by: \`$IDENTITY\`"
            echo "- SPDX SBOM attestation: present and verified"
            echo ""
            echo "Verify it yourself:"
            echo '```'
            echo "cosign verify --certificate-identity '$IDENTITY' \\"
            echo "  --certificate-oidc-issuer '$ISSUER' '$REF'"
            echo '```'
            echo ""
            echo "> A passing verification proves this artefact came out of this pipeline."
            echo "> It does **not** prove the code is safe — see docs/02-build-integrity.md."
          } >> "$GITHUB_STEP_SUMMARY"

      - name: Record the verifiable reference
        run: |
          echo "$REF" > image-ref.txt
          echo "$(cat image-ref.txt)"
      - uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882 # v4.4.3
        with:
          name: release-evidence
          path: |
            sbom.spdx.json
            verify-signature.json
            image-ref.txt
          retention-days: 90

  # ---------------------------------------------------------------------------
  # Deploy runs in a protected GitHub Environment. That is what puts
  # `:environment:production` into the OIDC `sub` claim, which is what the prod
  # trust policy requires — so a push to main cannot reach prod without passing the
  # environment's reviewers.  (T1)
  deploy:
    needs: [build, attest]
    if: vars.AWS_ENABLED == 'true'
    runs-on: ubuntu-24.04
    environment: production
    permissions:
      contents: read
      id-token: write
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
        with: { persist-credentials: false }

      - uses: aws-actions/configure-aws-credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502 #
v4.0.2
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-accounts-api-deploy
          aws-region: ${{ env.AWS_REGION }}
          role-session-name: deploy-${{ github.run_id }}
          role-duration-seconds: 900

      # Digest, never tag. Kyverno rejects tag-only references at admission anyway
      # (policy/kyverno/policies/require-image-digest.yaml) — this is the same rule
      # applied one step earlier so the failure is a red pipeline, not a red cluster.
      - name: Roll out by digest
        run: |
          set -euo pipefail
          aws eks update-kubeconfig --name banking-shared-ap-south-1 --region "$AWS_REGION"
          kubectl -n accounts set image deploy/accounts-api \
            api="${{ needs.build.outputs.image }}@${{ needs.build.outputs.digest }}"
          kubectl -n accounts rollout status deploy/accounts-api --timeout=180s

  # ---------------------------------------------------------------------------
```

```
  # Makes the skip legible instead of silent. A workflow whose production half
  # quietly does nothing is how a team ends up believing in a control it does not have.
  deploy-skipped-notice:
    needs: [build, attest]
    if: vars.AWS_ENABLED != 'true'
    runs-on: ubuntu-24.04
    steps:
      - run: |
          {
            echo "### Deploy: skipped"
            echo ""
            echo "\`AWS_ENABLED\` is not set, so the ECR push and EKS rollout did not run."
            echo "The build, SBOM, signing and verification above are real and ran against GHCR."
            echo "To exercise the AWS path, create the roles in \`iam/\` and set the repository"
            echo "variable \`AWS_ENABLED=true\`."
          } >> "$GITHUB_STEP_SUMMARY"
```